RunClub Hub - Project Report

Project overview and use of useEffect, Redux, and Axios

Project overview

RunClub Hub is a full-stack running-club application. Members can browse running events, register or sign in, create events, and leave or manage feedback.

Technology and architecture

- Frontend: React 18 and Vite, with React Router for page navigation.
- Shared event and feedback state: Redux Toolkit with React-Redux.
- Authentication state: React Context and localStorage; it is separate from Redux.
- Backend: Node.js and Express, with MongoDB/Mongoose for persistence.
- HTTP communication: Axios, configured through one shared client.

Main user flows

- Browse events and filter by event type.
- Register or log in; the server returns a signed token.
- Signed-in users can create events and submit feedback.
- Authors can delete their own events or feedback; the API checks ownership.

Frontend routes

The application routes are / (events), /login, /register, and a not-found route. Route definitions are in client/src/App.jsx.

Scope: this report documents the project structure and source-code usage visible in the client and server folders.

React effects and Redux

Where asynchronous page loading and shared state are implemented

useEffect: two application call sites

- client/src/pages/EventsPage.jsx, around line 20: on page load, GET /events is requested. Loading, error, and returned event data are dispatched to Redux. The effect uses an isCurrent guard during cleanup.

- client/src/components/FeedbackList.jsx, around line 23: GET /events/:eventId/feedbacks is requested when the list is shown or eventId changes. Results and request status are dispatched to Redux; cleanup prevents stale dispatches.

In both cases, useEffect is used for fetching data in response to component lifecycle or dependency changes. Other network calls are triggered by user actions instead.

Redux setup and state

- client/src/main.jsx, around line 13: the Redux Provider makes the store available throughout the React app.

- client/src/redux/store.js: configureStore combines the events and feedback reducers.

- client/src/redux/eventsSlice.js: stores events, loading, and error state; actions replace, add, remove, and update request status.

- client/src/redux/feedbackSlice.js: stores feedback, loading, and error state; actions load, add, remove, or clear feedback.

Components connected to Redux

- EventsPage reads event data with useSelector and dispatches load results and status.

- EventForm dispatches a newly created event; EventCard reads feedback and dispatches event and feedback removal.

- FeedbackList reads and updates feedback; FeedbackForm dispatches newly submitted feedback.

Authentication is managed separately in client/src/context/AuthContext.jsx, rather than in a Redux slice.

Axios and API request map

How the browser communicates with the Express API

Shared Axios client

client/src/api/axiosClient.js creates the shared Axios instance. Its base URL is VITE_API_URL when configured, otherwise http://localhost:5000/api. A request interceptor adds the token from localStorage as an Authorization header when one exists.

Requests and their source files

- POST /auth/login and POST /auth/register - client/src/context/AuthContext.jsx. Successful responses update React auth state and localStorage.

- GET /events - client/src/pages/EventsPage.jsx. The response populates the Redux events slice.

- POST /events - client/src/components/EventForm.jsx. The response is added to Redux.

- DELETE /events/:id - client/src/components/EventCard.jsx. On success, the event and its feedback are removed from Redux.

- GET /events/:eventId/feedbacks - client/src/components/FeedbackList.jsx. The response is saved in Redux for the selected event.

- POST /events/:eventId/feedbacks - client/src/components/FeedbackForm.jsx. The returned feedback is added to Redux.

- DELETE /feedbacks/:feedbackId - client/src/components/FeedbackList.jsx. The deleted feedback is removed from Redux.

End-to-end pattern

For initial page data, useEffect starts an Axios GET request and dispatches the result or error to Redux. For form submissions and deletions, event handlers make Axios requests, then update Redux after success. Redux supplies shared data to the UI; Axios handles HTTP transport.

Relevant source files

- Client: client/src/pages/EventsPage.jsx, client/src/components/FeedbackList.jsx, client/src/api/axiosClient.js, client/src/redux/store.js, client/src/redux/eventsSlice.js, client/src/redux/feedbackSlice.js.

- Server API: server/routes/authRoutes.js, server/routes/eventRoutes.js, server/routes/feedbackRoutes.js.

Generated November 10, 2026.